# From Research Code to CRAN: LLM-Assisted Refactoring of orthoMTL for Survival Endpoints

An agent, a rejected package, and what a skill file did (not) add

**Kevin Vervier** **Nadia Solovieff** **Gunther Jansen**

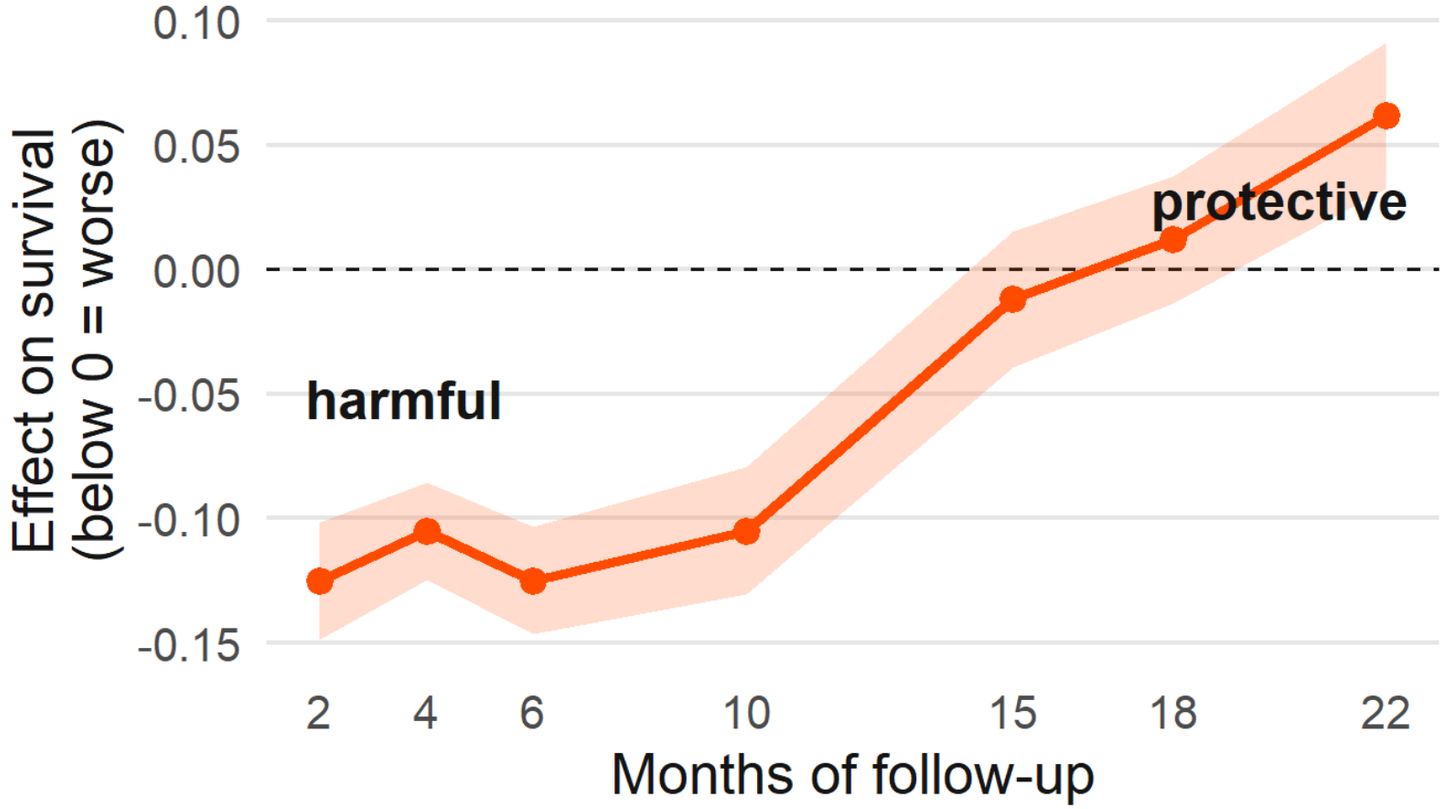
Novartis · R/Pharma 2026 · 20 October 2026

# A biomarker that changes its mind

In a late-stage oncology trial, one biomarker looked **unrelated to survival**.

One Cox coefficient **≈ 0**.

The curve is **orthoMTL**: one estimate per landmark.




Schematic of one biomarker's effect. Shape is illustrative.

**Harmful early. Protective late.** The two halves cancelled out.

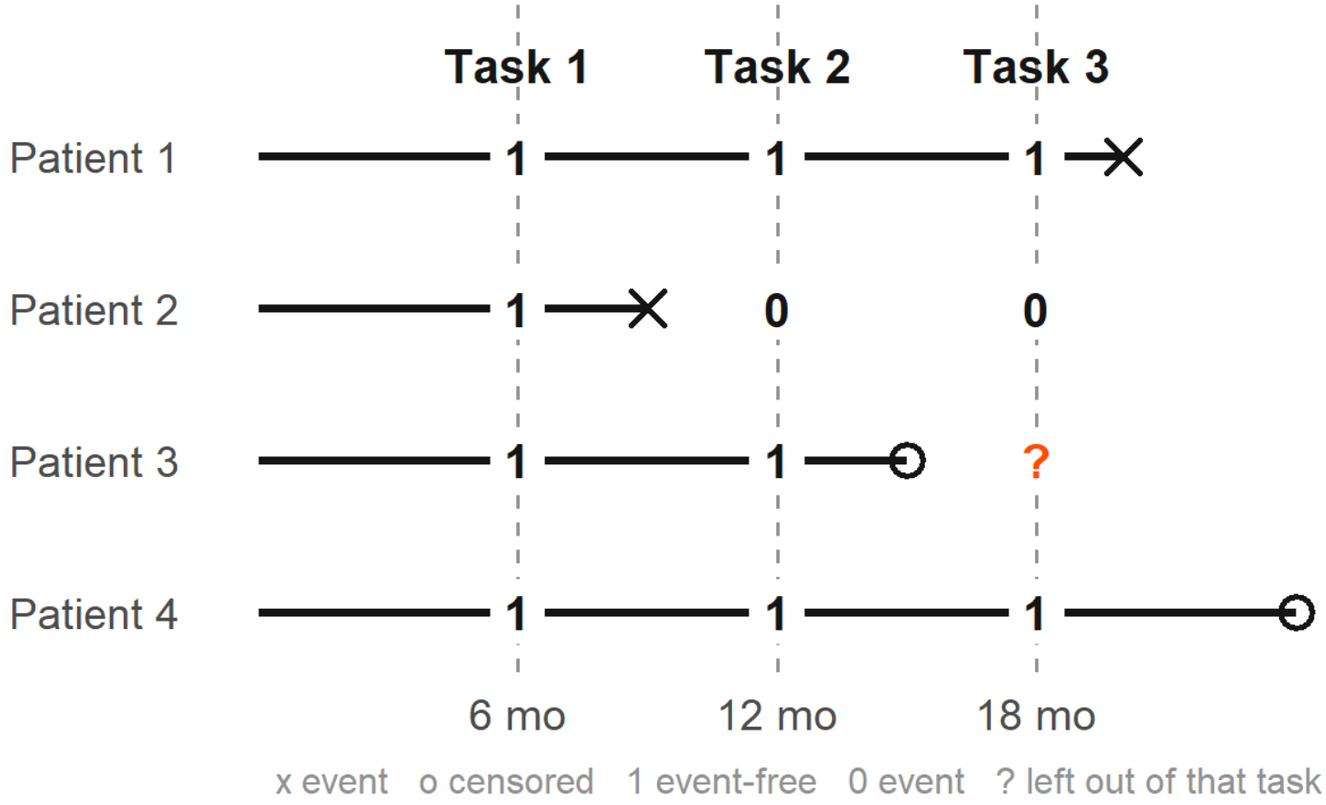
**So how does orthoMTL get one estimate per landmark?**

NOVARTIS

# orthoMTL: don't average over time

Cut follow-up into **landmarks**: one yes/no task each.

Fit them **together**, each with **its own effect**.




Framing after MTLSA (Li et al., KDD 2016).

`github.com/Novartis/orthoMTL` (on CRAN, 2026)

NOVARTIS

# From PhD code to CRAN

Twelve-year-old research code.

An **AI agent**, a language model that edits files and runs commands itself, did the packaging, **in a chat, with me in the loop**.

Writing unit tests for the **logistic loss**, it found a **real bug** in my own 2014 gradient: labels were ±1, the code used the 0/1 formula. I checked it by hand. It was real and fixed.

Then every automated check went green:

`R CMD check`: **0 errors | 0 warnings**

**Tests pass. Checks pass. Bundle built. Submit.**

# Then CRAN rejected us. Twice.

1. An invalid `LICENSE` field.
2. A hard-coded `set.seed()`.

No automated check caught either. **A CRAN volunteer** caught both.

Both were **knowledge the agent didn't have**. I could have written it down and handed it over.

**People call that a skill file.**

# What is a skill file?

A **Markdown file of instructions** the agent reads before it starts.

**Would one have got us through CRAN in one shot?**

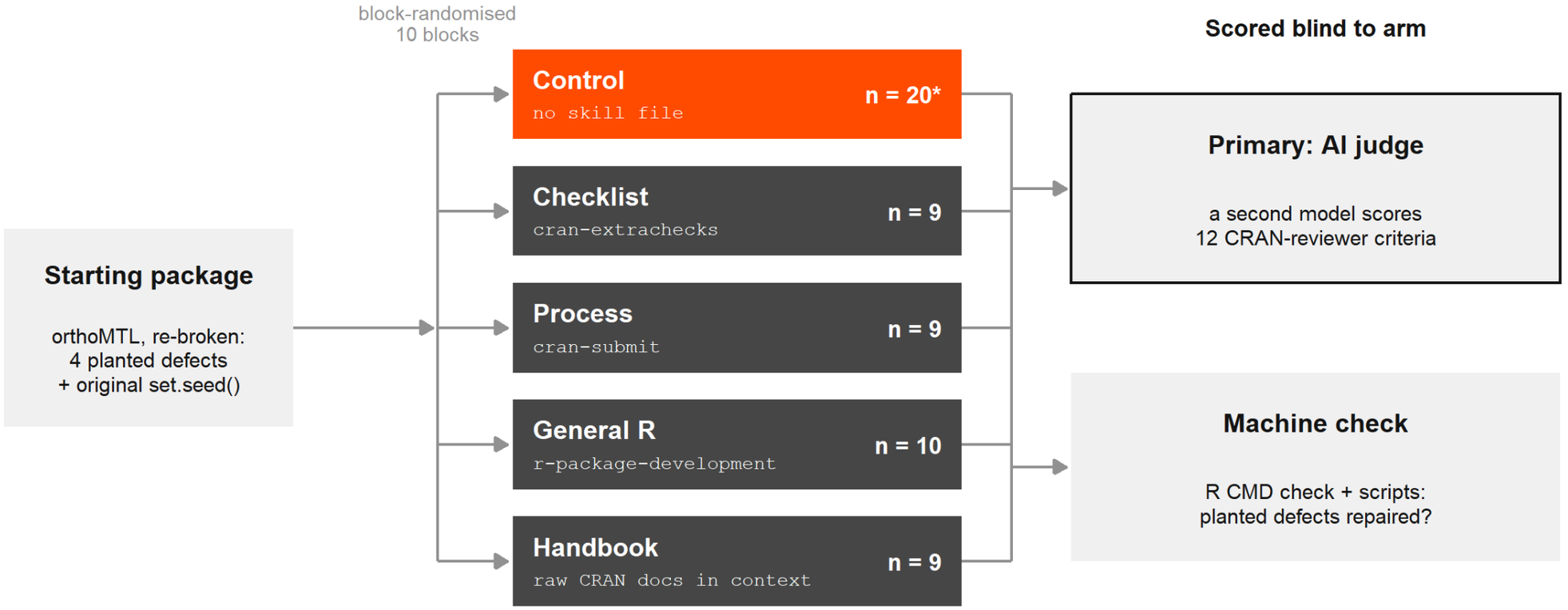
```
# CRAN submission notes for this package

## 1. Do not set a seed to a specific number
##    inside a function
- Search every file in `R/` for `set.seed(`.
- Give the function a `seed = NULL` argument ...
```

Excerpt (abridged): the file I later wrote from our own two rejections.

# We ran it like a trial

**Every run:** Pre-registered | One agent, no human | Frozen prompt | Fresh copy each run | CRAN docs in every arm




60 runs planned, 3 lost to API rate limits, 57 analysed. * includes 10 runs whose skill file never arrived.

NOVARTIS

# The result

# 57/57

runs fixed all four planted defects, **control included**

`R CMD check` clean in **57/57**. Submission-ready, with **no human in the loop at all**.

The agent did just as well with **no skill file at all**.

# Why the control did so well

The agent runs `R CMD check` and reads its own failures.

A machine check beats a checklist: it reads **your** package, not a generic one.

**Skill files add little where a tool already tells the truth.**

# Where the check is silent: `set.seed()`

`set.seed(42)` inside four exported functions: our second rejection.

`R CMD check` says nothing.

Here, the arms **did** differ.

Post-hoc, exploratory.

| Arm | `seed = NULL` fix |
|---|---|
| Control | **0** / 20 |
| Checklist | 0 / 9 |
| Process | **7** / 9 |
| General R | 0 / 10 |
| Handbook | **8** / 9 |

# Then I handed it the answer

A file quoting the reviewer, word for word. Run afterwards.

**0/10**

control

**10/10**

with the file

After unblinding, labels known. A ceiling check: the file contains the answer.

NOVARTIS

# The part I did not expect

My primary measure, the AI judge, had **12 criteria**. Ten never fired.

Our maintainer email, **unchanged in all 57 runs**:
fine **10** times, a problem **47** times.

**Every “finding” was my instrument. Not my arms.**

# What I am *not* claiming

- **Not** “skill files are useless”: this is the machine-checkable part of CRAN prep, with the docs at hand
- **N = 1**: one package, one agent model, a judge from the **same vendor**. Not powered; n was set by budget
- **A protocol deviation**: one arm’s file never reached the model. Those 10 runs joined the control

# Take home

**1.** Effects that change over time hide in averages → `orthoMTL`

**2.** Agents do well what a **machine** can check. Invest in **executable checks** first.

**3.** I set out to measure skill files. My judge **measured itself**.

**Machine checks are cheap. Judgement isn’t.**

`github.com/Novartis/orthoMTL` · Vervier et al. 2014 · Juric et al. 2026 · full references in backup

# Backup

NOVARTIS

# Backup: what the five arms were

| Arm | Skill file | Source | Runs | Repo code |
|---|---|---|---|---|
| Control | none | n/a | **20** (10 + 10, see below) | A0 (+ A4) |
| Checklist | `cran-extrachecks` | posit-dev/skills | 9 | A1 |
| Process | `cran-submit` | CoryMcCartan/agent-skills | 9 | A2 |
| General R | `r-package-development` | posit-dev/skills | 10 | A3 |
| Handbook | raw handbook text | CRAN docs, next slide | 9 | A5 |

Every arm, **the control included**, also had the same offline CRAN docs on disk. 57 of 60 planned runs completed; 3 ended on API rate limits.

Agent: Claude Opus 5.5 (`claude-opus-5-5`) in Claude Code 2.1.282–284 (auto-updated mid-experiment). Judge: Claude Sonnet 5, same vendor family as the agent. Skill files were delivered as auto-loaded `CLAUDE.md`, not through the on-demand Skill tool, which the agent did not have.

# Backup: the handbook every arm could read

Snapshot captured **2026-09-18**, identical across all 60 runs.

| Source | URL |
|---|---|
| CRAN Cookbook | contributor.r-project.org/cran-cookbook |
| CRAN Repository Policy | cran.r-project.org/web/packages/policies.html |
| CRAN Submission Checklist | cran.r-project.org/web/packages/submission_checklist.html |

The Cookbook is built from real CRAN re-submissions and co-authored by a member of the CRAN team: the closest public thing to the reviewers' own knowledge. The handbook arm loaded this text directly into context; the other arms were told where it was.

# Backup: the hedges, in their own words

| Finding | Source |
|---|---|
| Skills “often provide no benefit, and can even hurt performance while incurring additional token costs” | SkillDelta, Xu et al. 2026 |
| Human-authored skills +16.2 pp; **LLM-authored skills give no measurable gain** | SkillAxe, Gautam et al. 2026 |
| Agents used retrieved skills 70% of the time yet showed **no performance gain** where the corpus lacked good skills for the domain | SkillFlow, Li et al. 2026 |
| Self-generated skills landed **below** the no-skills baseline (−8 to −11.5 pp) | SkillsBench, Li et al. 2026 |

My null is the same shape, in a domain none of them covered.

# Backup: the counterweight

The headline positive result is **SkillsBench**: curated skills lift pass rate 33.9% → 50.5% (**+16.6 pp**) across 87 tasks and 18 model–harness configurations. I am not disputing it. Three things make it a different question from mine:

- **8 domains**: natural science, media, cybersecurity, industrial, finance, office, software engineering, maths/OR. **No R, no packaging, no CRAN**
- Tasks are admitted only if **“significantly easier with Skills”**; no-separation tasks are rejected as low-signal
- Skills are drawn from the **top quality quartile** (10.1/12 vs ecosystem mean 6.2/12); the authors call this “an optimistic scenario”

Their own domain gradient points my way: **software engineering +11.6 pp**, second-lowest of eight, against natural science **+28.8 pp**, “domains with stronger pretraining and tooling coverage benefit less.”

# Backup: references (1/2)

### Package and method

- orthoMTL: https://github.com/Novartis/orthoMTL
- Predecessor `orthopen`: https://github.com/kevinVervier/orthopen
- Vervier K., Mahé P., Vert J.-P. (2014). *Benchmark of structured machine learning methods for microbial identification from mass-spectrometry data*. Springer. Method origin
- Li Y. et al. (2016). *A Multi-Task Learning Formulation for Survival Analysis*. KDD. Survival-as-multi-task framing (MTLSA), framing behind the landmark diagram
- Juric D. et al. (2026). *Annals of Oncology*. Clinical application

### Skill files under test

- Posit `skills`: https://github.com/posit-dev/skills
- McCartan C., `agent-skills`: https://github.com/CoryMcCartan/agent-skills

**CRAN documentation**: Cookbook, Repository Policy and Submission Checklist: URLs on the landing page

# Backup: references (2/2)

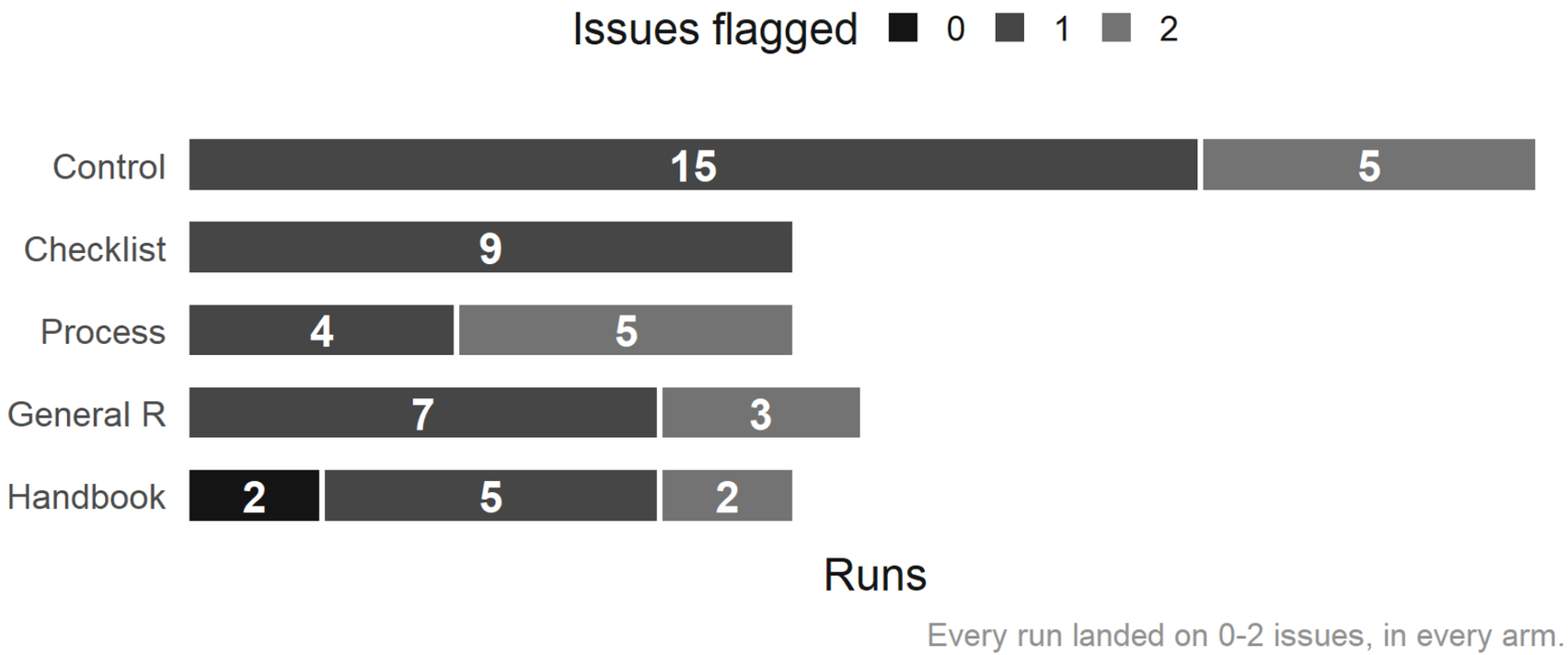
**Prior measurement of agent skills**


- Xu A. et al. (2026). *When Does a Skill Add Value? Task-Conditional Gain Prediction for Selective Skill Use*. arXiv:2609.32274
- Gautam S., Radhakrishna A., Gulwani S. (2026). *SkillAxe: Sharpening LLM-Authored Agent Skills Through Evaluation-Guided Self-Refinement*. arXiv:2606.10546
- Li F., Tagkopoulos P., Tagkopoulos I. (2026). *SkillFlow: Scalable and Efficient Agent Skill Retrieval System*. COLM 2026, arXiv:2504.06188
- Li X. et al. (2026). *SkillsBench: Benchmarking How Well Agent Skills Work Across Diverse Tasks*. arXiv:2602.12670

# Backup: the three ways we checked correctness

| Check | What it proves |
|---|---|
| Against the old validated package | identical coefficients on 5 fixed cases |
| Flip every label | answer must flip; `0.62` → `0.000` |
| Simulated data, known truth | recovers the right features, 10/10 seeds |

# Backup: judge-flagged issues per arm




Issues the AI judge flagged, per run. Every run, in every arm, landed on **0–2**, and the flags came from two criteria the judge scored inconsistently on unchanged text (the judge slide).

NOVARTIS

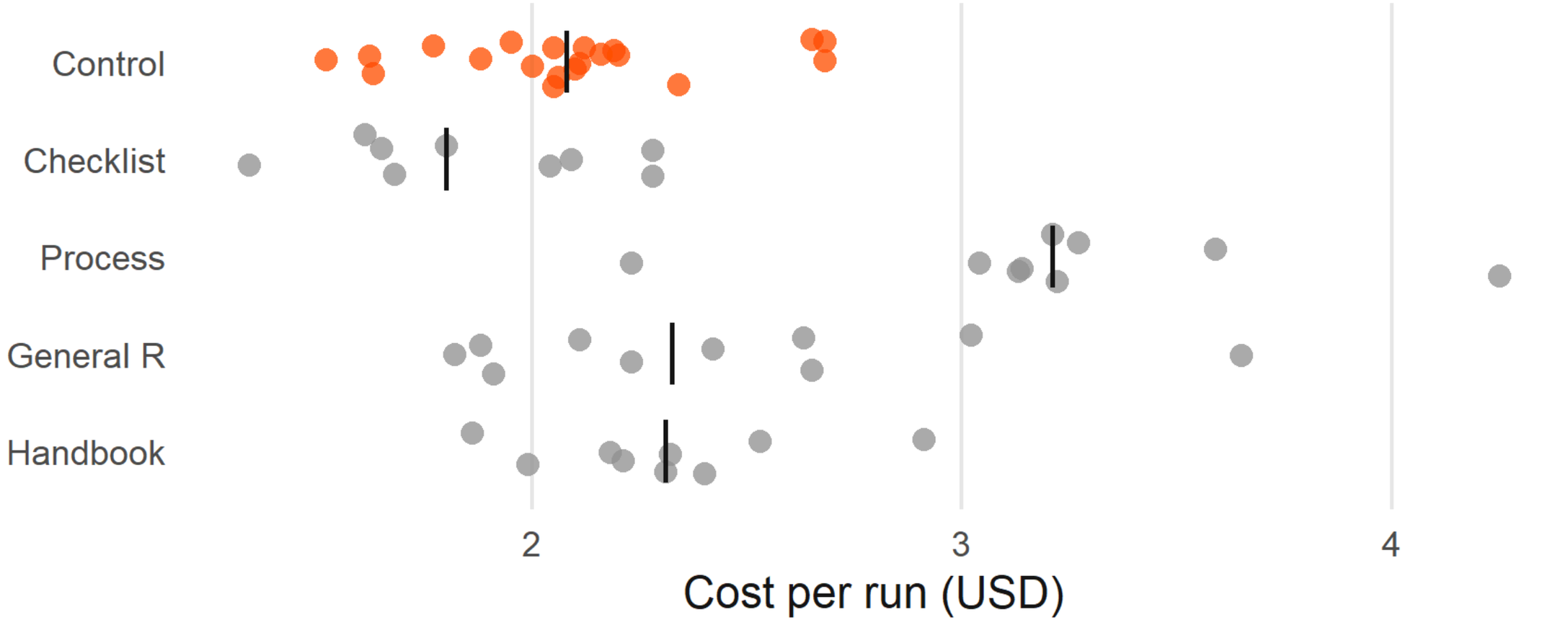
# Backup: our own maintainer address

Even a noisy judge can be right once. It flagged our own maintainer address in 47 of 57 trees:

`kevin.vervier@novartis.com`

CRAN requires **a person, not a mailing list**, kept current and reachable "**without any form of filtering**". A corporate account is one re-org away from failing all three. **Not a violation today. Changing in 0.2.0 anyway.**

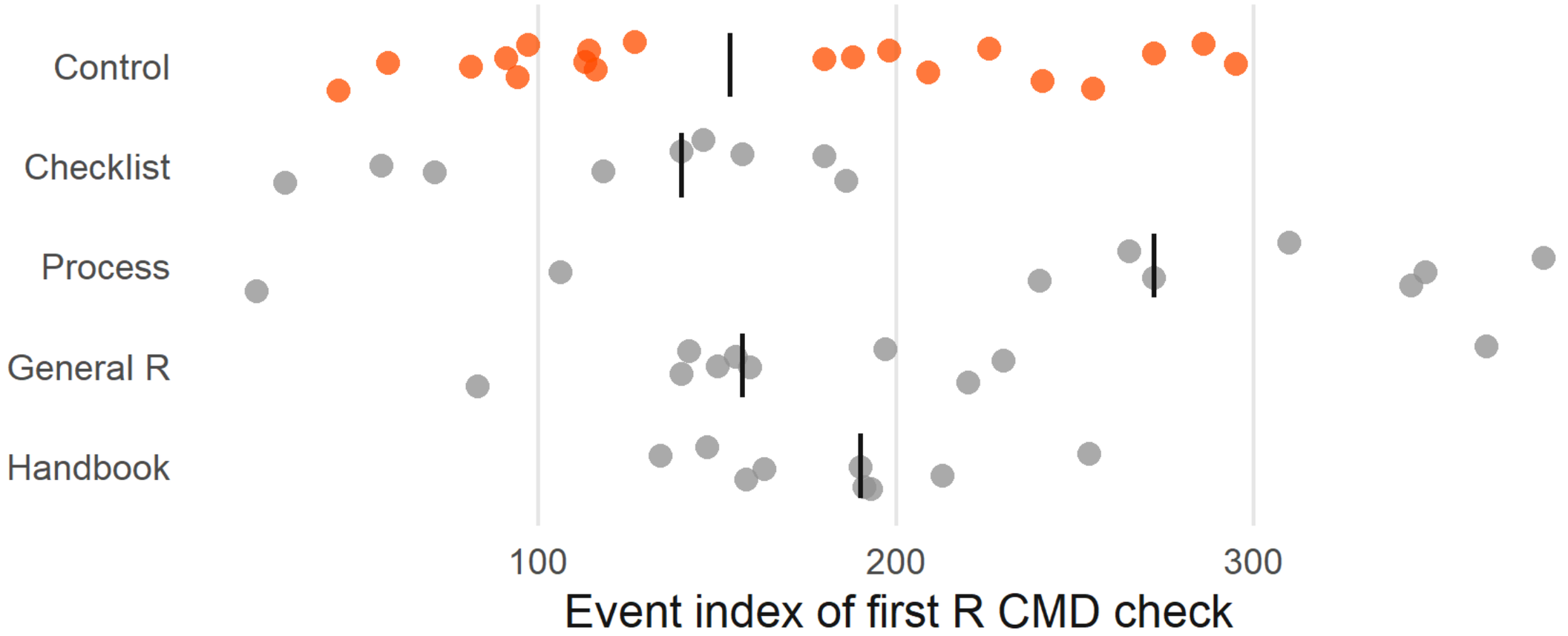
# Backup: cost per run




Dots = runs; bar = median.

NOVARTIS

# Backup: when did the agent first run `R CMD check`?




Dots = runs; bar = median.

NOVARTIS

# Backup: what I would change

- Score **R01 (`set.seed()`)** with a rule that also sees seeds passed as default arguments
- Plant more defects only a **CRAN volunteer** could catch, and calibrate the judge on them
- A **positive control** in the blinded run, plus a pre-flight check that each arm's file reaches the model
- A **handicapped arm** with the CRAN docs removed: if it does worse, the measurement has teeth
- Judge each package several times, not once
- More packages, more models

# Backup: the four planted defects

| Defect | Caught by `R CMD check`? | Repaired |
|---|---|---|
| `NAMESPACE` deleted | yes, check cannot even run | 57/57 |
| `usethis` LICENSE template vs GPL-3 DESCRIPTION | **no**, this is rejection **R02** | **57/57** |
| DESCRIPTION metadata (`Authors@R`, title case) | yes | 57/57 |
| Unused declared `Imports` | yes | 57/57 |

Three of four are machine-visible. Our other real rejection, **R01, the hard-coded** `set.seed()`, was left in the starting package as a fifth defect that `R CMD check` does not report; it is on the next slide.

Three of four machine-visible is the result's main caveat: this measures the easy half of CRAN preparation.

# Backup: the hard-coded seed (R01)

| Arm | n | Canonical fix (`seed = NULL`) | Default kept, RNG restored | Still hard-coded |
|---|---|---|---|---|
| Control | 20 | 0 | 10 | 10 |
| Checklist | 9 | 0 | 8 | 1 |
| Process | 9 | 7 | 1 | 1 |
| General R | 10 | 0 | 7 | 3 |
| Handbook | 9 | 8 | 0 | 1 |

Completed runs; descriptive, no test. “Default kept”: `seed = 42` stays but a helper restores the user’s RNG state: arguably acceptable, not what the reviewer asked for. Control includes the 10 runs whose hindsight file never arrived.

**Re-scored after unblinding.** The original rule only matched `set.seed(<literal>)`, so it scored **zero hits on the starting package itself**. The fixed rule reads package files only and never touches the blind map: the same code at first unblinding gives this same table.

# Backup: positive control, run afterwards (R01)

The hindsight file was written from CRAN's own two rejection messages, installed, and checked to reach the model (first-call context +1,055 tokens over its paired control). Then a fresh control and the hindsight arm were run as 10 interleaved pairs, **after unblinding, labels known**, on a newer Claude Code (2.1.294).

| Arm | n | Canonical fix (`seed = NULL`) | Default kept | Still hard-coded |
|---|---|---|---|---|
| Fresh control | 10 | 0 | 6 | 4 |
| Hindsight file delivered | 10 | 10 | 0 | 0 |

Two extra hindsight runs without a surviving partner (rate limits) were also canonical: 12 of 12. This is a ceiling check, not a skill-file result: the file contains the answer. It shows the seed defect is learnable from text in context, which the blinded experiment could not show. Not pooled with the 57; no judge panel was run on these trees. `R CMD check` was identical across arms: no errors or warnings in any of the 22 trees. Scripts and scores: `supplementary/`.

NOVARTIS